

Top Tips & Tricks from Day 3

cloning anything, organizing everything, and the two files that teach a folder how to behave

Day 3 moved us from *getting Claude Code working* to *working well inside it*. The big idea: a project is just a folder, and two special documents — **CLAUDE.md** and **skills** — let that folder tell Claude who you are and how you like things done.

REPOS & GITHUB

- **Cloning is a core skill — three ways.** Download the ZIP from the green Code button; *git clone* in the terminal; or open a session in the parent folder and just ask Claude to clone the URL. Then the catch everyone hits: start a **new session inside the cloned folder** — cloning and working are two different sessions.
- **git pull makes a repo a living document.** If we add files after you cloned, don't re-download — ask Claude to "do a git pull" and only the changes come down. That's what makes a repo more like a Google Doc than a ZIP.
- **GitHub is a context goldmine for your discipline.** Search "github repo *your topic*": all of Shakespeare from MIT, Berkeley courses, papers shipping their data and scripts, and the ed-tech tools you already use (TimelineJS, Scalar) — finally openable instead of black boxes.
- **Judge a repo before you clone it.** Stars, forks, and watchers; a trusted institution; and age — never install something brand-new that no one has ever used. Then ask Claude itself: *look around — how safe do you think this is?*

SESSIONS, MODELS & STAYING ORGANIZED

- **Pick the model deliberately.** New top-tier **Fable 5** is powerful but burns credits faster and refuses readily near sensitive topics. For nearly everything here, **Opus 4.8** (or Sonnet) is the workhorse. Check */usage*; if it feels slow, turn thinking down before switching.
- **Decoder ring.** Session = chat = task = thread — **each is one context window**. And in Code and Cowork, a "project" is simply **a folder on your computer**: the unit of work.
- **Group your chats.** Recents settings → *Group by project* sorts chats by folder; or use a chat's three-dots menu → *Move to group* for custom groups ("writing zone"). Cosmetic — it organizes you, not the model.
- **Start new sessions early and often — context rot is real.** The start and the most recent turns carry the most weight; quality sags in the middle. Don't wait for auto-compact, and don't live in one thread for months — that's the road to a diluted system prompt and strange behavior.

TWO DOCUMENTS THAT TEACH A FOLDER

- **CLAUDE.md — things that are always true.** A file named CLAUDE.md at a project's root is read automatically every session. Proof from class: one line at the bottom — "end everything with a limerick" — and every reply arrived in verse. Use it for who you are, how you want output styled, what the project is. Make one in every folder: *"please create a CLAUDE.md in this folder that..."* (Edits load in *new* sessions.)
- **Skills — things that are sometimes true.** A SKILL.md is "just a prompt ... a conditional prompt": a reusable description of how you like a recurring process done, living in **.claude/skills/<name>/. Claude loads just the names + descriptions — a map of your prompt library — and proposes the right one when your request matches. If you'd paste the same prompt twice, make it a skill; it can even point at a script you trust so known-good code runs every time. (Your daily handouts? A formatting skill.)**

WORKING PATTERNS WORTH STEALING

- **Run any worked example on your own material.** Drag your syllabus into the example's *inputs/*, three-dots → *copy path*, paste it and say "run these operations on just this file." Or go big: "loop through all twenty syllabi in inputs." A million tokens is a lot.
- **Build a harvesting workflow.** Keep a folder where you toss PDFs as you find them, plus a standing CLAUDE.md instruction to convert and summarize it on request — then "harvest my downloads." For web pages: select-all, copy, paste, "make this a clean markdown doc."
- **PDFs are a view layer.** Do the thinking in text the model reads natively — markdown, or LaTeX when equations matter — and render to PDF only at the end. Text in, text out: no hallucinated equations.

Next up — Day 4: APIs and MCPs. API keys are "little secret passwords" that let services talk to each other: pull documents in, push results out, bring other models (Claude + Gemini on the same blackboard photo) into the loop and compare. That — plus MCPs — is today.